Lab Work Assignment 6: Streamlit-Based Interactive Model Analysis Dashboard

Course: ML Operations / ML Engineering

Objective:

The objective of this lab is to design and implement an interactive data science dashboard using Streamlit. The dashboard must integrate with previously developed training pipelines and experiment tracking systems (e.g., MLflow) and provide tools for dataset exploration, model error analysis, and model prediction interpretability.

Students are expected to build a modular, production-style Streamlit application that enables structured inspection of datasets, comparison of experiment runs, visualization of model errors, and explanation of model predictions using interpretability techniques such as Grad-CAM or LIME.

Resources

- Streamlit Official Documentation. https://docs.streamlit.io/
- Grad-CAM Original Paper (Selvaraju et al., 2017). https://arxiv.org/abs/1610.02391
- LIME Official Documentation. https://lime-ml.readthedocs.io/en/latest/
- Matplotlib Documentation. https://matplotlib.org/stable/index.html
- Plotly Python Documentation. https://plotly.com/python/
- Scikit-learn Metrics Documentation. https://scikit-learn.org/stable/modules/model_evaluation.html
- Interpretable Machine Learning Book (Molnar). https://christophm.github.io/interpretable-ml-book/

Submission Deadline:
Submit your completed project and lab report by xx.xx.2026.

Part 1: Project Setup and Application Architecture

[Task] Environment Setup

Install Streamlit and required dependencies in your Poetry-managed environment. Required components may include:

- streamlit
- mlflow
- torch / tensorflow (depending on your framework)
- matplotlib / plotly
- lime or grad-cam implementation

Ensure the project remains reproducible and isolated.

[Task] Application Structure

Design a clean and modular project structure. Separate the following concerns:

- Data loading utilities
- MLflow interaction utilities
- Model loading and inference utilities
- Visualization utilities
- Streamlit UI components

Avoid placing all logic in a single app.py file. The dashboard must follow a structured architecture similar to a lightweight frontend layer over your ML pipeline.

[Task] Streamlit Navigation

Implement multiple tabs using Streamlit (e.g., st.tabs()). At minimum, the application must include the following tabs:

1. Dataset Exploration
2. Error Analysis
3. Prediction & Explainability

The UI must be deterministic and reproducible (avoid uncontrolled randomness unless explicitly labeled).

Part 2: Dataset Exploration Tab

Objective: Provide structured inspection of the dataset.

[Task] Dataset Overview
Display high-level dataset statistics:

- Number of samples
- Number of classes
- Class distribution (bar chart)
- Train/validation/test split sizes

Use visualizations (matplotlib or plotly).

[Task] Sample Inspection
Implement functionality to:

- Select a specific dataset split
- Select a sample by index
- Display the image (or input data)
- Display its label

If working with image data, render images directly. If working with tabular data, render feature values in a structured table.

[Task] Filtering
Allow filtering by class label to inspect only samples belonging to a specific category.

The dataset tab must not retrain any model. It must strictly perform analysis and visualization.

Part 3: Model Error Analysis Tab (MLflow Integration)

Objective: Analyze model failures using experiment tracking data.

[Task] MLflow Integration
Connect to your MLflow tracking server. Retrieve:

- Available experiments
- Available runs within an experiment
- Logged metrics
- Logged artifacts

The user must be able to:

- Select an experiment
- Select a specific run

[Task] Error Extraction
For the selected run:

- Load the logged model artifact
- Run inference on the test dataset
- Identify misclassified samples

[Task] Visual Error Inspection
Display:

- Confusion matrix
- Per-class error counts
- Individual misclassified examples

For each misclassified example display:

- Input image
- True label
- Predicted label
- Prediction confidence

Optional (additional points):
Implement sorting of errors by confidence or by predicted class. All computations must be reproducible and tied to the selected MLflow run.

Part 4: Model Prediction & Explainability Tab

Objective: Provide inference and model interpretability tools.

[Task] Run Selection
Allow selecting an MLflow run that contains a trained model artifact.

[Task] Inference on Dataset Samples
Allow the user to:

- Select a dataset sample
- Run inference
- Display predicted label and probability distribution

[Task] Inference on Uploaded Images (if using image classification)
Allow users to upload a custom image file and run model inference on it.

[Task] Explainability Methods
Integrate at least one model interpretability technique:

Option A: Grad-CAM (for CNN-based image models)
Option B: LIME (for image or tabular models)

For Grad-CAM:

- Generate class activation heatmap
- Overlay heatmap on input image

For LIME:

- Generate explanation for the selected prediction
- Visualize important regions or features

Display both raw prediction and explanation result side by side.

[Task] Optional (additional points):
Allow selecting which class to explain (not only the top prediction).

Part 5: Code Quality and Engineering Practices

[Task] Configuration Management
Store configuration parameters (e.g., MLflow URI, dataset path, model type) in a configuration file. Avoid hard-coded paths inside the Streamlit application.

[Task] Logging
Use Python logging to record:

- Model loading
- MLflow queries
- Inference execution
- Errors

Do not use print() statements.

[Task] Error Handling
Ensure the dashboard handles:

- Missing MLflow runs

- Corrupted artifacts
- Incorrect model formats
- Invalid uploaded files

The application must fail gracefully with informative messages.

[Task] Dependency Management
Use Poetry for dependency isolation.

[Task] Version Control
Use GitHub to manage your repository.
Provide meaningful commit messages reflecting incremental development of features.

Part 6: Lab Report

Introduction:
Explain the purpose of interactive dashboards in ML systems and their role in model monitoring and debugging.

Architecture Description:
Describe the structure of your Streamlit application and how it interacts with MLflow and the trained models.

Dataset Analysis:
Summarize insights obtained from the dataset exploration tab.

Error Analysis:
Discuss typical failure patterns observed in misclassified samples. Include screenshots of the confusion matrix and example errors.

Explainability Results:
Present examples of Grad-CAM or LIME explanations and interpret what the model focuses on.

Engineering Reflection:
Discuss design decisions, architectural trade-offs, and limitations of your implementation.

Code Submission:
Submit your complete repository including:

- Streamlit application
- Utility modules
- Configuration files
- Screenshots of dashboard tabs
- Clear README with setup and execution instructions

The application must be runnable using a single command (e.g., streamlit run app.py).

Grading Criteria

Architecture and Modularity:
Clean separation between UI, data handling, model inference, and MLflow utilities.

MLflow Integration:
Correct retrieval of runs, artifacts, and metrics.

Error Analysis Implementation:
Accurate identification and visualization of misclassified samples.

Explainability Integration:
Correct implementation and interpretation of Grad-CAM or LIME.

Code Quality and Engineering Practices:
Use of configuration files, logging, dependency management, and version control.

Report Quality:
Clarity, depth of technical explanation, and critical analysis of results.